

Astaroth: A scientific computing framework for accelerating stencil computations.

Touko Puro¹, Johannes Pekkilä¹, Miikka S. Väisälä², Oskar Lappi³, Matthias Rheinhardt¹, Hsien Shang⁴, and Maarit Korpi-Lagg¹

¹ Aalto University, Finland ² University of Oulu, Finland ³ University of Helsinki, Finland ⁴ Academia Sinica Institute of Astronomy and Astrophysics, Taiwan ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Patrick Diehl](#)  

Reviewers:

- [@rfj82982](#)
- [@streeve](#)
- [@atmyers](#)

Submitted: 20 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Stencil computations¹ are one of the bedrocks of high-performance scientific simulations, forming the core of many partial differential equation (PDE) and numerical linear algebra solvers. In recent years, GPUs have become the primary compute platform for data-parallel applications in high-performance computing, and it is difficult to run large simulations without them. Astaroth is a GPU framework for stencil computations, that has been developed to address this problem of scalable scientific computing. Astaroth provides its own domain specific language (DSL), in which researchers can express such computations without having to focus on technical implementation details. It can run efficiently both on CUDA- and HIP-based environments, and for testing purposes on CPUs. While stencils are the core of Astaroth, it also accelerates other operations like reductions (e.g. sums), simple ray-tracing, and integrates with libraries performing GPU-accelerated Fourier transforms, all of which are important for simulations on structured grids. Astaroth is optimized for multiphysics use cases and has primarily been used for turbulent astrophysical plasma simulations.

Statement of need

Due to the ubiquity and expense of stencil computations, a common library for implementing them efficiently on GPUs is needed, more so because high-performance codes require portability across multiple different computing platforms. Such a library is both needed for accelerating existing codes and writing new ones. Thus, its API has to enable good integration with existing and easy writing of new code.

Astaroth strives to be this library, encapsulating the core steps and structures of stencil computations in its domain specific language (DSL)², which enables researchers in different domains to write their stencil computations in a compact and obvious way. This encapsulation allows Astaroth to provide the needed execution platform for different application cases, predominantly for partial differential equation (PDE) solvers, but also for image processing and numerical linear algebra.

Multiphysics simulations in particular pose computationally challenging stencil calculations due to the need of holding values of many fields in working memory at the same time, and

¹Stencil computations are computations on structured grids where a given point is updated using a fixed neighborhood pattern. Examples are convolutions in image processing and convolutional neural networks, and different schemes for spatial derivatives like the finite-difference method.

²For a simple example of the Astaroth DSL and a program using it, see the advection solver example in <https://github.com/ToxPuro/astaroth/tree/9295cbc4a649881e94af83971b52bfc167a72e69/samples/advection-example>.

36 accelerating them was the original need for which Astaroth was developed. As an example of
37 this use case, Astaroth has been used to accelerate the widely used astrophysics framework
38 Pencil Code, for which existing solutions were not fit.

39 State of the field

40 Several approaches have been proposed to improve the performance, portability, and
41 productivity dimensions of stencil computations. Single-process approaches include domain-
42 specific languages (DSLs), e.g., Halide (Ragan-Kelley et al., 2013), PolyMage (Mullapudi
43 et al., 2015), Delite (Sujeeth et al., 2014), and Lift (Steuwer et al., 2017), and general
44 parallel processing abstractions, e.g., Kokkos (Trott et al., 2021) and RAJA (Beckingsale et al.,
45 2019). The primary benefit of a DSL is that assumptions about the structure of computations
46 can be made to improve the performance of generated code while maintaining a high-level
47 representation for the user. Performance-portability is typically augmented with automated
48 tuning and algorithm selection (e.g., PATUS (Christen et al., 2011), PARTANS (Lutz et al.,
49 2013)). These approaches are also adopted in Astaroth, which introduces a DSL, a code
50 generator, and implements automated tile-size optimization. A distinctive feature of Astaroth
51 is its algorithmic specialization for cache-constrained use cases in multiphysics, where the
52 working set required to update interdependent physical fields is too large to fit into on-chip
53 caches. This is addressed by reordering the computations into stages of higher and lower cache
54 reuse, thereby enabling better utilization of on-chip caches (Pekkilä et al., 2025). Astaroth
55 not only consider stencils in isolation, but also their combinations with other operations
56 inside the same kernel, such as the automatic fusion of distributed reductions with other
57 computations.

58 Methods to improve the performance-portability-productivity dimensions in multiprocess
59 applications include topology-aware workload distribution and abstractions for distributed
60 operations. Chapel (Callahan et al., 2004), Charm++ (Kalé & Krishnan, 1993) improve
61 the performance-portability-productivity dimensions by providing general-purpose distributed
62 programming models, whereas Cactus (Goodale et al., 2003), PETSc (Mills et al., 2021)
63 implement scientific computation toolkits. Further productivity advances can be gained with
64 domain-specialized frameworks, which provide ready-made solutions for specific use cases
65 and a simplified application programming interface. Astaroth also belongs to this class of
66 frameworks. Examples closest to Astaroth are Parthenon (Grete et al., 2023) and AMReX
67 (Zhang et al., 2019). Both are frameworks for distributed adaptive mesh refinement (AMR),
68 where Parthenon uses Kokkos as the compute backend and AMReX provides compute features
69 with parallel function wrappers and user-written C++ lambdas. In contrast to these projects,
70 Astaroth's distributed abstraction layer focuses on structured grid computations without mesh
71 refinement, and can thus make simplifying assumptions about the underlying data-movement
72 patterns to better address on-chip data movement, batching, and data-processing pipelines.
73 Similar to Parthenon and others (Pearson, 2021), Astaroth implements its modification of
74 topology-aware domain decomposition and rank reordering for improved portability across
75 systems, and performs fused packing to alleviate communication overheads. Furthermore,
76 Astaroth implements a task scheduler for compute and communication tasks (Lappi, 2021).

77 In the field, Astaroth stands out as a CUDA/HIP stencil-computing framework focused on
78 addressing the performance-productivity trade-off in cache-heavy multiphysics applications with
79 a domain-specific language, automated tile-size optimization, and topology handling, taking
80 ownership of data structures and movement throughout the computational science pipeline.
81 This enables holistic optimizations of full scientific workflows and offers a lower barrier of entry
82 to experimentation with optimization techniques spanning traditionally decoupled tasks (e.g.,
83 extensive kernel fusion of operations across the stack), which would not be practical with
84 libraries utilizing opaque submodules for compute and communication.

Software design

Astaroth consists of three main components: 1) `acc`, a compiler and runtime system for a domain-specific language (DSL) for stencil computations, 2) an API for executing stencil applications on multi-GPU platforms, and 3) a standalone PDE solver. Below, we present a quick overview of these components. Extensive documentation is available at ([The Astaroth developers, 2026](#)).

`acc` compiler and runtime system

Astaroth has a DSL for stencil-based computations, designed to be used by domain scientists without having to consider technical implementation details. The main operations, such as stencils, are written in a declarative syntax, and the kernels that use them are written in an imperative syntax.³ The implementation is left to Astaroth's DSL compiler `acc`, which applies a number of specialized optimizations. An especially important optimization is the unrolled and reordered computation of all required stencils at the start of the kernels, which enables instruction-level parallelism and efficient usage of caches ([Pekkilä, 2026](#)). In addition to stencils, the DSL supports two other operations: 1) multi-GPU reductions – which are commonly needed for stencil-based solvers; and 2) simplified distributed ray-tracing, where rays cannot change directions and are restricted to move through neighbouring grid points – which is necessary for simulations incorporating radiative transfer ([Heinemann et al., 2006](#)). Astaroth's DSL also includes a standard library, providing, inter alia: derivative operators used in PDE solvers, implemented for generally spaced Cartesian, spherical or cylindrical grids; and Poisson solvers, e.g. for self-gravity ([Krasnopolsky et al., 2026](#)).

`acc` transpiles the DSL source into CUDA or HIP source code, which is further compiled into machine code using a native CUDA or HIP compiler. The program thus produced is executed in the `acc` runtime system, which further optimizes the kernels by autotuning the thread block sizes for kernel execution. `acc` also supports run-time compilation, because run-time configuration parameters may change the evaluation of conditional statements, thereby changing the branches taken at run-time. With run-time compilation, `acc` compiles for a given configuration only those parts of the DSL source that will be executed. The information of what code gets executed also allows Astaroth to optimize run-time behaviour more precisely, e.g. memory allocations or communication patterns.

Multi-GPU runtime system and API

In the DSL, using the `ComputeSteps` language construct, users can define a list of compute steps specifying a sequence of kernels and boundary conditions. Kernels defined in `ComputeSteps` may be fused to reduce memory reads. Based on the overall domain decomposition and the stencils' data access patterns, `acc` infers the dependency relationships between the steps, and constructs a directed acyclic graph (DAG) of dependent tasks. Each step is split into many tasks, one per region: there are 26 smaller regions at each subdomain's boundaries, which are dependent on communicated data from neighbors; and one big region at the core of the subdomain, which is not. Communication tasks are inserted where needed.

Astaroth's task scheduler executes these DAGs, asynchronously launching computation and communication tasks when prerequisite tasks are completed. This improves performance in communication-bound cases, especially for higher process counts ([Lappi, 2021](#)). For fast data transfers and to support all possible hardware, both GPU-to-GPU remote direct memory access (RDMA) and CPU-to-CPU communication are supported.

This runtime system can be accessed through Astaroth's runtime API. The API is C-ABI compatible, supporting foreign function interfaces to external applications written in any

³In declarative programming, computations are defined by describing what the results look like; in imperative programming, by describing the steps to perform.

131 programming language. The API is organized into two layers: the Device layer and the Grid
132 layer. The Device layer provides access to single-GPU functionality, such as: moving data
133 between CPU and GPU, launching kernels, and loading/storing snapshots from/to disk. The
134 Grid layer provides access to multi-GPU functionality, such as: executing DAGs, distributed
135 initialization, and distributed loading/storing of snapshots. Other special functionality is also
136 provided through the API, such as distributed Fourier transforms.

137 Solver

138 Astaroth also includes a standalone finite-difference PDE solver (Pekkilä et al., 2022), which
139 takes full advantage of the DSL and the multi-GPU API, and can be used to write new
140 simulation models. The solver scales to thousands of GPUs with a weak scaling efficiency
141 >90% (Pekkilä, 2026) and also works as a testbed for performance research.

142 The solver uses an astrophysical magnetohydrodynamical setup (`acc-runtime/samples/mhd_modular`)
143 by default, but can be configured to run any DSL code. The samples directory also includes
144 other production-ready setups, e.g. `tfm-mpi` for the test-field method (Pekkilä, 2026).

145 The solver handles distributed initial conditions, domain decomposition, simulation diagnostics,
146 and logging. It is also designed to react to a number of events, such as NaNs in the simulation
147 data, simulation time limits, and a stop signal given through the file system. The directory
148 `analysis/` contains Python-based data analysis tools, which can be used to process and work
149 with the data produced by the standalone solver.

150 Research impact statement

151 Astaroth has already been used in a number of papers as the core PDE-solver, mainly for
152 astrophysical plasma simulations (Gent et al., 2026; Väisälä et al., 2021, 2023), but also in
153 seismology (Ladino et al., 2025). Additionally, it has been used for research on performance
154 optimization methods (Pekkilä et al., 2017, 2025; Pekkilä, 2026), communication techniques
155 (Lappi, 2021; Pekkilä et al., 2022), compiler techniques (Pekkilä, 2019; Puro, 2023) and
156 other topics (Puro & Pohjonen, 2025; Yokelson et al., 2024). We expect that the recent
157 GPU-acceleration of Pencil Code, which was done by embedding Astaroth's DSL and runtime
158 system into it, will increase the number of Astaroth users. The associated speedup factor
159 of 20-60 (Pekkilä et al., 2022) will enable more realistic astrophysical simulations in a wide
160 range of use cases from modelling small-scale dynamos (Warnecke et al., 2025) to processes
161 producing primordial gravitational waves and their propagation (Roper Pol et al., 2020).

162 Acknowledgements

163 We acknowledge the contributions of all developers and early users of Astaroth who have
164 been instrumental in its evolution. These include Petr Bém, Jörn Warnecke, Frederick Gent,
165 Ruben Krasnopolsky, Wei-Wen Li, Mordecai Mac Low, Chun-Fan Liu, Man Hei Li, Tzu-Chun
166 Hsu and Indrani Das. We acknowledge the computational resources and services provided
167 by CSC — IT Center for Science, the Aalto Science-IT project, ASIAA High-Performance
168 Computing, and National Center for High-Performance Computing (NCHC), National Applied
169 Research Laboratories (NARLabs) in Taiwan, the Oak Ridge Leadership Computing Facility
170 at the Oak Ridge National Laboratory, and resources from LUMI-G through the Euro-HPC
171 joint undertaking. Furthermore, we appreciate the important technical assistance provided by
172 CSC, by people like Fredrik Robertsén and others. The development of Astaroth has received
173 funding from the Academy of Finland, ReSolVE Centre of Excellence, Grant/Award Number:
174 307411; The European Research Council, the European Union's Horizon 2020 research and
175 innovation program, project UniSDyn, Grant/Award Number: 818665; KAUTE Foundation,
176 Grant/Award Numbers: 20240173 and 20250154; Research Council of Finland, project MomEnt,
177 Grant/Award Number: 373416. The authors acknowledge support for the CompAS Project

from the Institute of Astronomy and Astrophysics, Academia Sinica (ASIAA), the Academia Sinica grant AS-IAIA-114-M01, and the National Science and Technology Council (NSTC) in Taiwan through grants 112-2112-M-001-030, 113-2112-M-001-008, and 114-2112-M-001-001-; the International Collaboration and Cooperation grant for COSMAGG that supports the exchanges between Taiwan and Finland: 113-2927-I-001-513-, 114-2927-I-001-506-, and Research Council of Finland project 359462. MSV thanks the support of Jenny and Antti Wihuri Foundation and Finnish Cultural Foundation in his doctoral thesis work, and therefore the early development in Astaroth prototype.

AI usage disclosure

AI tools have not been used in any step of software creation, documentation or in the authoring of this paper.

References

- Beckingsale, D. A., Burmark, J., Hornung, R., Jones, H., Killian, W., Kunen, A. J., Pearce, O., Robinson, P., Ryujin, B. S., & Scogland, T. R. (2019). RAJA: Portable performance for large-scale scientific applications. *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3hpc)*, 71–81. <https://doi.org/10.1109/p3hpc49587.2019.00012>
- Callahan, D., Chamberlain, B. L., & Zima, H. P. (2004). The cascade high productivity language. *Proceedings of the Ninth International Workshop on High-Level Parallel Programming Models and Supportive Environments, 2004. Proceedings.*, 52–60. <https://doi.org/10.1109/HIPS.2004.1299190>
- Christen, M., Schenk, O., & Burkhart, H. (2011). PATUS: A code generation and autotuning framework for parallel iterative stencil computations on modern microarchitectures. *Proceedings of the 2011 IEEE International Parallel & Distributed Processing Symposium*, 676–687. <https://doi.org/10.1109/IPDPS.2011.70>
- Gent, F. A., Mac Low, M.-M., Korpi-Lagg, M. J., Puro, T., & Rheinhardt, M. (2026). Asymptotic behaviour of galactic small-scale dynamos at modest magnetic prandtl number. *The Astrophysical Journal Letters*, 1000(2), L40. <https://doi.org/10.3847/2041-8213/ae4c41>
- Goodale, T., Allen, G., Lanfermann, G., Massó, J., Radke, T., Seidel, E., & Shalf, J. (2003). The cactus framework and toolkit: Design and applications. In G. Goos, J. Hartmanis, J. Van Leeuwen, J. M. L. M. Palma, A. A. Sousa, J. Dongarra, & V. Hernández (Eds.), *High performance computing for computational science — VECPAR 2002* (pp. 197–227). Springer. https://doi.org/10.1007/3-540-36569-9_13
- Grete, P., Dolence, J. C., Miller, J. M., Brown, J., Ryan, B., Gaspar, A., Glines, F., Swaminarayan, S., Lippuner, J., Solomon, C. J., Shipman, G., Junghans, C., Holladay, D., Stone, J. M., & Roberts, L. F. (2023). Parthenon—a performance portable block-structured adaptive mesh refinement framework. *The International Journal of High Performance Computing Applications*, 37(5), 465–486. <https://doi.org/10.1177/10943420221143775>
- Heinemann, T., Dobler, W., Nordlund, Å., & Brandenburg, A. (2006). Radiative transfer in decomposed domains. *Astronomy & Astrophysics*, 448(2), 731–737. <https://doi.org/10.1051/0004-6361:20053120>
- Kalé, L. V., & Krishnan, S. (1993). CHARM++: A portable concurrent object oriented system based on C++. *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*, 91–108. <https://doi.org/10.1145/165854.165874>

- 224 Krasnopolsky, R., Puro, T., Li, W.-W., Shang, H., Väisälä, M. S., Mac Low, M.-M., Rheinhardt,
225 M., & Korpi-Lagg, M. (2026). Iterative poisson solvers for self-gravity with the GPU code
226 astaroth. *arXiv Preprint arXiv:2605.03563*. <https://doi.org/10.48550/arXiv.2605.03563>
- 227 Ladino, O., Coelho, B. S., Wyzykowski, A. B. V., & Soares, L. (2025). Acoustic wave
228 modeling using the astaroth framework. *Empowering the Energy Shift: The Role of HPC*
229 *in Sustainable Innovation, 2025*, 1–3. <https://doi.org/10.3997/2214-4609.2025643013>
- 230 Lappi, O. (2021). A task scheduler for astaroth, the astrophysics simulation framework.
231 *Master's Thesis*. <https://urn.fi/URN:NBN:fi-fe2021050428868>
- 232 Lutz, T., Fensch, C., & Cole, M. (2013). PARTANS: An autotuning framework for stencil
233 computation on multi-GPU systems. *ACM Transactions on Architecture and Code*
234 *Optimization*, 9(4), 59:1–59:24. <https://doi.org/10.1145/2400682.2400718>
- 235 Mills, R. T., Adams, M. F., Balay, S., Brown, J., Dener, A., Knepley, M., Kruger, S. E.,
236 Morgan, H., Munson, T., Rupp, K., Smith, B. F., Zampini, S., Zhang, H., & Zhang, J.
237 (2021). Toward performance-portable PETSc for GPU-based exascale systems. *Parallel*
238 *Computing*, 108, 102831. <https://doi.org/10.1016/j.parco.2021.102831>
- 239 Mullanpudi, R. T., Vasista, V., & Bondhugula, U. (2015). Polymage: Automatic optimization
240 for image processing pipelines. *ACM SIGARCH Computer Architecture News*, 43(1),
241 429–443. <https://doi.org/10.1145/2694344.2694364>
- 242 Pearson, C. (2021). *Movement and placement of non-contiguous data in distributed GPU*
243 *computing* [PhD thesis, University of Illinois at Urbana-Champaign]. [https://hdl.handle.](https://hdl.handle.net/2142/110511)
244 [net/2142/110511](https://hdl.handle.net/2142/110511)
- 245 Pekkilä, J. (2019). Astaroth: A library for stencil computations on graphics processing units.
246 *Master's Thesis*. <https://urn.fi/URN:NBN:fi:aalto-201906233993>
- 247 Pekkilä, J. (2026). *Graphics processors in high-performance computing: Practice and experience*
248 *in the simulation of astrophysical magnetohydrodynamics and turbulence* [PhD thesis,
249 Aalto University School of Science]. <https://urn.fi/URN:ISBN:978-952-64-3160-4>
- 250 Pekkilä, J., Lappi, O., Robertsén, F., & Korpi-Lagg, M. J. (2025). Stencil computations on
251 amd and nvidia graphics processors: Performance and tuning strategies. *Concurrency and*
252 *Computation: Practice and Experience*, 37(12-14), e70129. [https://doi.org/10.1002/cpe.](https://doi.org/10.1002/cpe.70129)
253 [70129](https://doi.org/10.1002/cpe.70129)
- 254 Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Käpylä, P. J., & Anjum, O. (2017). Methods
255 for compressible fluid simulation on GPUs using high-order finite differences. *Computer*
256 *Physics Communications*, 217, 11–22. <https://doi.org/10.1016/j.cpc.2017.03.011>
- 257 Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Rheinhardt, M., & Lappi, O. (2022). Scalable
258 communication for high-order stencil computations using CUDA-aware MPI. *Parallel*
259 *Computing*, 111, 102904. <https://doi.org/10.1016/j.parco.2022.102904>
- 260 Puro, T. (2023). Programmatic and efficient gpu acceleration of cpu stencil codes. *Master's*
261 *Thesis*. <http://hdl.handle.net/10138/569388>
- 262 Puro, T., & Pohjonen, A. (2025). GPU acceleration of average gradient method for solving
263 partial differential equations. *Proceedings of the Second SIMS EUROSIM Conference on*
264 *Modelling and Simulation, SIMS EUROSIM 2024*. <https://doi.org/10.3384/ecp212.066>
- 265 Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., & Amarasinghe, S. (2013).
266 Halide: A language and compiler for optimizing parallelism, locality, and recomputation
267 in image processing pipelines. *Acm Sigplan Notices*, 48(6), 519–530. [https://doi.org/10.](https://doi.org/10.1145/2491956.2462176)
268 [1145/2491956.2462176](https://doi.org/10.1145/2491956.2462176)
- 269 Roper Pol, A., Mandal, S., Brandenburg, A., Kahnashvili, T., & Kosowsky, A. (2020).
270 Numerical simulations of gravitational waves from early-universe turbulence. *Physical*

- 271 Review D, 102(8), 083512. <https://doi.org/10.1103/physrevd.102.083512>
- 272 Steuwer, M., Rummel, T., & Dubach, C. (2017). LIFT: A functional data-parallel IR for
273 high-performance GPU code generation. *Proceedings of the 2017 IEEE/ACM International*
274 *Symposium on Code Generation and Optimization*, 74–85. [https://doi.org/10.1109/CGO.](https://doi.org/10.1109/CGO.2017.7863730)
275 2017.7863730
- 276 Sujeeth, A. K., Brown, K. J., Lee, H., Rompf, T., Chafi, H., Odersky, M., & Olukotun,
277 K. (2014). Delite: A compiler architecture for performance-oriented embedded domain-
278 specific languages. *ACM Transactions on Embedded Computing Systems*, 13(4s), 1–25.
279 <https://doi.org/10.1145/2584665>
- 280 The Astaroth developers. (2026). *Astaroth documentation*. [https://toxypuro.github.io/](https://toxypuro.github.io/astaroth/)
281 astaroth/. <https://toxypuro.github.io/astaroth>
- 282 Trott, C. R., Lebrun-Grandié, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R.,
283 Harvey, E., Hollman, D. S., Ibanez, D., & others. (2021). Kokkos 3: Programming model
284 extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*,
285 33(4), 805–817. <https://doi.org/10.1109/tpds.2021.3097283>
- 286 Väisälä, M. S., Pekkilä, J., Käpylä, M. J., Rheinhardt, M., Shang, H., & Krasnopolsky, R.
287 (2021). Interaction of large-and small-scale dynamos in isotropic turbulent flows from
288 GPU-accelerated simulations. *The Astrophysical Journal*, 907(2), 83. [https://doi.org/10.](https://doi.org/10.3847/1538-4357/abceca)
289 3847/1538-4357/abceca
- 290 Väisälä, M. S., Shang, H., Galli, D., Lizano, S., & Krasnopolsky, R. (2023). Exploring the
291 formation of resistive pseudodisks with the GPU code astaroth. *The Astrophysical Journal*,
292 959(1), 32. <https://doi.org/10.3847/1538-4357/acfb00>
- 293 Warnecke, J., Korpi-Lagg, M. J., Rheinhardt, M., Viviani, M., & Prabhu, A. (2025). Small-scale
294 and large-scale dynamos in global convection simulations of solar-like stars. *Astronomy &*
295 *Astrophysics*, 696, A93. <https://doi.org/10.1051/0004-6361/202451085>
- 296 Yokelson, D., Lappi, O., Ramesh, S., Väisälä, M. S., Huck, K., Puro, T., Norris, B., Korpi-Lagg,
297 M., Heljanko, K., & Malony, A. D. (2024). SOMA: Observability, monitoring, and in situ
298 analytics for exascale applications. *Concurrency and Computation: Practice and Experience*,
299 36(19), e8141. <https://doi.org/10.1002/cpe.8141>
- 300 Zhang, W., Almgren, A., Beckner, V., Bell, J., Blaschke, J., Chan, C., Day, M., Friesen, B.,
301 Gott, K., Graves, D., Katz, M., Myers, A., Nguyen, T., Nonaka, A., Rosso, M., Williams, S.,
302 & Zingale, M. (2019). AMReX: A framework for block-structured adaptive mesh refinement.
303 *Journal of Open Source Software*, 4(37), 1370. <https://doi.org/10.21105/joss.01370>